



**UNIVERSIDAD
SERGIO ARBOLEDA**

Gramatica Terra

Lenguajes de programación

**Yslen Natalia Moreno Gutierrez
Santiago Patiño Sarmiento**

*Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 09 de septiembre de 2026*

Índice

1. Propósito de este documento	4
1.1. Qué es una gramática	4
1.2. Notación empleada	4
2. Estructura del programa	4
3. Sentencias básicas	5
4. Estructuras de control	6
4.1. El caso intermedio se escribe con dos palabras	6
4.2. Cómo se alinean condiciones y bloques	7
4.3. Terminación explícita	7
5. Instrucciones del dominio	7
6. Expresiones y tuberías	8
6.1. Dos niveles de expresión	8
6.2. La tubería admite dos estilos	8
6.3. Operaciones	9
7. Precedencia de operadores	9
7.1. Por qué esto produce la precedencia correcta	10
7.2. Asociatividad de la potencia	10
7.3. Pertenencia sin encadenamiento	10
8. Elementos primarios	11
8.1. Funciones sin palabras reservadas	11
8.2. Indexación	11
9. Definiciones léxicas	12
9.1. Palabras reservadas	12
9.2. Literales y operadores	12
9.3. Tokens generales	13
9.4. Elementos descartados	13

10.Resolución de conflictos léxicos	13
11.Verificación	14

1. Propósito de este documento

La gramática de Terra está implementada en ANTLR v4, en el archivo `gramaticas/Terra.g4`. Este documento la presenta en notación BNF extendida, que es la forma estándar de describir la sintaxis de un lenguaje con independencia de la herramienta que se use para implementarla.

Cada regla aparece acompañada de una explicación de qué construcción describe y, cuando la decisión no es obvia, de por qué está escrita así.

1.1. Qué es una gramática

Una gramática es un conjunto de reglas que define exactamente qué cadenas de texto pertenecen a un lenguaje y cuáles no. Tiene dos clases de símbolos.

Los **terminales** son los símbolos que aparecen literalmente en el programa: la palabra `fijo`, el signo `+`, un número. No se descomponen en nada más pequeño.

Los **no terminales** son nombres de construcciones que se definen a partir de otros símbolos. `expresion` es un no terminal: no se escribe nunca en un programa, pero describe todo lo que puede ocupar el lugar de una expresión.

Analizar un programa consiste en verificar si puede derivarse desde el símbolo inicial, que en Terra es `programa`, aplicando las reglas. Si se puede, el programa es válido y el proceso deja como resultado un árbol de análisis.

1.2. Notación empleada

Símbolo	Significado
<code>::=</code>	A se define como B
<code> </code>	Alternativa: una opción u otra
<code>*</code>	Cero o más repeticiones
<code>+</code>	Una o más repeticiones
<code>?</code>	Opcional: cero o una vez
<code>()</code>	Agrupación
<code>'token'</code>	Palabra reservada o símbolo literal
<code>MAYUSCULAS</code>	Token léxico (terminal)
<code>minúsculas</code>	Regla sintáctica (no terminal)

2. Estructura del programa

```
programa ::= ( NL | sentencia )* EOF
```

```
sentencia ::=
```

```

    declaracion_variable
  | asignacion
  | mostrar_stmt
  | si_stmt
  | mientras_stmt
  | para_stmt
  | guardar_stmt
  | graficar_stmt
  | expresion_stmt

```

`bloque ::= ( NL | sentencia )*`

Un programa es una secuencia de sentencias y saltos de línea que termina en el fin del archivo. El token `EOF` es obligatorio: obliga al parser a consumir toda la entrada, de modo que un programa con basura al final se rechaza en vez de analizarse a medias.

En Terra el salto de línea (NL) no se descarta como espacio en blanco, porque es el terminador de sentencia. Por eso **programa** y **bloque** lo mencionan de forma explícita.

La forma `( NL | sentencia )*` dice que un bloque es una sucesión de elementos, cada uno de los cuales es o bien un salto de línea o bien una sentencia. La alternativa más natural, `( NL* sentencia )*`, obliga a que todo bloque termine en una sentencia y rechaza los programas con líneas en blanco antes del **fin**, algo perfectamente legítimo al escribir código.

La versión adoptada acepta líneas en blanco en cualquier posición y el evaluador simplemente ignora los nodos NL al recorrer el árbol.

3. Sentencias básicas

```

declaracion_variable ::=
    'fijo ' IDENTIFICADOR '=' expresion NL
  | 'var ' IDENTIFICADOR '=' expresion NL

```

`asignacion ::= IDENTIFICADOR '=' expresion NL`

`expresion_stmt ::= expresion NL`

```

mostrar_stmt ::=
    'mostrar' '(' ( expresion ( ',' expresion )* )? NL

```

(En `mostrar_stmt` el paréntesis de cierre se omite arriba por espacio; la regla real es `'mostrar' '(' (...)? ')' NL`.)

La distinción entre **fixo** y **var** es puramente sintáctica en la gramática: ambas alternativas tienen la misma forma. La diferencia de comportamiento la impone el evaluador, que marca la variable como mutable o inmutable en la tabla de símbolos.

mostrar acepta cero o más argumentos separados por comas, y los imprime en una sola línea separados por espacios.

4. Estructuras de control

```
si_stmt ::=
    'si' expresion ':' NL
    bloque
    ( 'sino' 'si' expresion ':' NL bloque ) *
    ( 'sino' ':' NL bloque ) ?
    'fin' NL
```

```
mientras_stmt ::=
    'mientras' expresion ':' NL
    bloque
    'fin' NL
```

```
para_stmt ::=
    'para' IDENTIFICADOR 'en' expresion ':' NL
    bloque
    'fin' NL
```

4.1. El caso intermedio se escribe con dos palabras

Muchos lenguajes crean una palabra reservada para el caso intermedio del condicional: **elif** en Python, **elsif** en Ruby. Terra usa la secuencia de dos tokens existentes, **sino** seguido de **si**.

La ventaja es que no hay que reservar una palabra más ni inventar un término artificial. La objeción posible es la ambigüedad: al encontrar **sino**, ¿se trata del caso intermedio o del caso por defecto? El analizador de ANTLR no tiene ese problema porque no decide con un solo token de anticipación: examina los tokens siguientes tanto como haga falta y ve si después de **sino** viene un **si** o directamente los dos puntos.

La comprobación práctica es que la gramática compila sin ninguna advertencia de ambigüedad.

4.2. Cómo se alinean condiciones y bloques

En `si_stmt`, ANTLR entrega todas las condiciones en una lista y todos los bloques en otra. El evaluador las recorre en paralelo: la condición i corresponde al bloque i . Cuando hay un bloque más que condiciones, ese bloque sobrante es el del `sino` final. Esta correspondencia por posición evita tener que etiquetar las alternativas en la gramática.

4.3. Terminación explícita

Los tres bloques cierran con `fin`. La alternativa sería usar la indentación, como Python, pero eso obliga al lexer a emitir tokens artificiales de sangría y desangría, con reglas delicadas para las líneas en blanco y los comentarios. Con `fin` el cierre es un token normal y la gramática se mantiene simple y legible.

5. Instrucciones del dominio

```
guardar_stmt ::= 'guardar' expr 'como' expr NL

graficar_stmt ::=
    'graficar' tipo_grafica expr ( NL* opcion_grafica )* NL

tipo_grafica ::=
    'barras' | 'lineas' | 'histograma'
    | 'dispersion' | 'caja'

opcion_grafica ::=
    'eje_x' IDENTIFICADOR
    | 'eje_y' IDENTIFICADOR
    | 'titulo' expr
    | 'leyenda' expr
    | 'guardar' 'como' expr
```

Las opciones de la gráfica van una por línea, en cualquier orden, y todas son opcionales. El fragmento `( NL* opcion_grafica )*` permite esa disposición: antes de cada opción se admiten saltos de línea.

Podría parecer que hay ambigüedad, porque tras cada opción viene un NL que también podría cerrar la instrucción. No la hay: el analizador mira qué token sigue al salto de línea. Si es una de las cinco palabras de opción, continúa el ciclo; si es cualquier otra cosa, cierra la instrucción.

Esa es también la razón de que los ejes se llamen `eje_x` y `eje_y` en vez de `x` e `y`: nombres tan cortos, convertidos en palabras reservadas, quedarían inutilizables

como nombres de variable en todo el lenguaje.

6. Expresiones y tuberías

```
expresion ::= tuberia
```

```
tuberia ::= expr ( NL* '>' NL* operacion )*
```

```
expr ::= o_logico
```

6.1. Dos niveles de expresión

Terra distingue **expresion** de **expr**, y esa separación resuelve un problema real.

expresion incluye las tuberías. **expr** es todo lo demás: aritmética, comparaciones, lógica, literales y variables, pero sin `>`.

Las operaciones de la tubería usan **expr**, no **expresion**. Sin esa restricción, en un programa como

```
datos
  > filtrar donde co2 > 0.0
  > ordenar por co2 desc
```

la expresión interna del **filtrar** intentaría absorber el `>` de la línea siguiente, porque los analizadores son voraces y consumen todo lo que la regla permita. El resultado sería una tubería anidada dentro del filtro, en vez de dos pasos consecutivos de la misma tubería.

Al escribir la condición del filtro como **expr**, el `>` queda fuera de su alcance y necesariamente pertenece a la tubería externa. Si alguna vez se quiere una tubería anidada de verdad, se escribe entre paréntesis, ya que `'( expresion )'` sí admite el nivel completo.

6.2. La tubería admite dos estilos

El fragmento `NL* '>' NL*` permite que el operador vaya al final de una línea o al comienzo de la siguiente. Ambas formas son válidas:

```
fijo a = datos >
  seleccionar [pais, co2]

fijo b = datos
  > seleccionar [pais, co2]
```

La segunda es la recomendada, porque alinea verticalmente los pasos del análisis.

6.3. Operaciones

```
operacion ::=
    'seleccionar' lista_columnas
  | 'filtrar' 'donde' expr
  | 'crear' IDENTIFICADOR '=' expr
  | 'renombrar' IDENTIFICADOR 'como' IDENTIFICADOR
  | 'ordenar' 'por' IDENTIFICADOR ( 'asc' | 'desc' )?
  | 'eliminar' 'duplicados'
  | 'eliminar' 'nulos' ( 'en' lista_columnas )?
  | 'rellenar' 'nulos' 'en' IDENTIFICADOR 'con' expr
  | 'convertir' IDENTIFICADOR 'como' tipo_dato
  | 'agrupar' 'por' lista_columnas
  | 'resumir' agregacion ( ',' NL* agregacion )*
  | 'limitar' expr
```

```
agregacion ::= IDENTIFICADOR '=' expr
```

```
lista_columnas ::=
    '[' IDENTIFICADOR ( ',' IDENTIFICADOR )* ']'
```

```
tipo_dato ::= 'numero' | 'decimal' | 'texto' | 'booleano'
```

Dos alternativas empiezan con `'eliminar'` y se distinguen por el token siguiente, `duplicados` o `nulos`. El analizador de ANTLR resuelve esa elección sin dificultad.

En `resumir`, el fragmento `( ',' NL* agregacion )*` permite repartir varias agregaciones en líneas distintas, que es como se escriben en la práctica:

```
|> resumir emisiones = suma(co2),
      promedio = media(co2-per-capita),
      registros = contar()
```

La conversión de tipo se escribe `convertir` `anio` `como` `numero`, reutilizando la palabra `como` que ya existe en `guardar` y en `renombrar`. Se descartó la preposición `a`, más natural en español, porque convertirla en palabra reservada impediría usar `a` como nombre de variable, que es de los más frecuentes en cualquier cálculo.

7. Precedencia de operadores

La precedencia no se declara en una tabla aparte: está codificada en la forma de las reglas. Cada nivel se define en términos del nivel inmediatamente superior, así que los operadores que se escriben en las reglas más profundas se evalúan primero.

```
o_logico    ::= y_logico ( '||' y_logico )*
y_logico    ::= igualdad ( '&&' igualdad )*
```

7.1 Por qué esto produce la precedencia correcta

```

igualdad      ::= relacional ( ( '=' | '!=' ) relacional ) *
relacional    ::= pertenencia
                ( ( '>' | '<' | '>=' | '<=' ) pertenencia ) *
pertenencia   ::= suma ( 'en' suma ) ?
suma          ::= termino ( ( '+' | '-' ) termino ) *
termino       ::= potencia ( ( '*' | '/' | '%' ) potencia ) *
potencia      ::= unario ( '^' potencia ) ?

unario ::=
    '-' unario
    | '!' unario
    | postfijo

postfijo ::= primario ( '[' expr ']' ) *

```

7.1. Por qué esto produce la precedencia correcta

Considérese $2 + 3 * 4$. La regla **suma** exige que sus operandos sean **termino**. Al analizar el 3, el parser está dentro de **termino** y encuentra el *****, que **termino** sí puede consumir. Así que $3 * 4$ se agrupa en un solo nodo antes de que **suma** lo reciba como operando derecho. El resultado es 14.

Para que fuese 20 tendría que existir una regla que permitiera al **+** agruparse primero, y no la hay: **suma** está por encima de **termino** en la jerarquía y siempre cede primero.

7.2. Asociatividad de la potencia

Los operadores escritos con un ciclo, como **termino** ::= **potencia** (***** **potencia**)*, asocian a la izquierda: $a * b * c$ se agrupa como $(a * b) * c$.

La potencia se escribe distinto, con recursión por la derecha: **potencia** ::= **unario** (**^** **potencia**)?. Como la regla se invoca a sí misma después del operador, la parte derecha se agrupa primero y $2 ^ 3 ^ 2$ equivale a $2 ^ (3 ^ 2)$, es decir 512. Esa es la convención matemática y la que siguen Python y la mayoría de calculadoras científicas.

7.3. Pertenencia sin encadenamiento

pertenencia ::= **suma** (**en** **suma**)? usa un interrogante en vez de un asterisco, así que el operador aparece a lo sumo una vez. Encadenarlo, como en **a en b en c**, no tiene un significado claro: **a en b** produce un booleano, y preguntar si un booleano pertenece a algo casi siempre delata un error. La gramática lo rechaza directamente.

8. Elementos primarios

```
primario ::=
    NUMERO
    | DECIMAL
    | TEXTO
    | BOOLEANO
    | 'nulo'
    | carga
    | llamada
    | IDENTIFICADOR
    | lista
    | '(' expresion ')'
```

```
carga ::= 'cargar' expr ( 'con' 'separador' expr )?
```

```
llamada ::= IDENTIFICADOR '(' ( expr ( ',' expr )* )? ')'
```

```
lista ::= '[' ( expr ( ',' expr )* )? ']'
```

8.1. Funciones sin palabras reservadas

Las funciones incorporadas (**media**, **suma**, **raiz**, **contar**) no aparecen en la gramática. La regla **llamada** acepta cualquier identificador seguido de paréntesis, y es el evaluador quien decide si ese nombre corresponde a una función conocida.

Esto tiene dos ventajas. Primero, agregar una función nueva no exige tocar la gramática ni regenerar el parser: basta registrarla en el evaluador. Segundo, el catálogo de palabras reservadas se mantiene corto, y cada palabra reservada es un nombre que el usuario pierde.

El precio es que un nombre de función mal escrito no se detecta al analizar sino al evaluar. Es un intercambio razonable: el mensaje de error resultante, **la funcion 'inventada' no existe**, es igual de claro.

8.2. Indexación

La regla del nivel postfijo permite encadenar índices, de modo que `matriz[0][1]` es válido sintácticamente. En esta entrega la indexación opera sobre listas y textos; desde la segunda servirá también para acceder a las columnas de una tabla.

9. Definiciones léxicas

El lexer convierte el texto en tokens antes de que actúe el parser. Estas son sus reglas, en el mismo orden en que aparecen en el archivo .g4.

9.1. Palabras reservadas

FIJO ::= 'fijo '	VAR ::= 'var '
SI ::= 'si '	SINO ::= 'sino '
FIN ::= 'fin '	MIENTRAS ::= 'mientras '
PARA ::= 'para '	EN ::= 'en '
MOSTRAR ::= 'mostrar '	CARGAR ::= 'cargar '
GUARDAR ::= 'guardar '	COMO ::= 'como '
CON ::= 'con '	SEPARADOR ::= 'separador '
SELECCIONAR ::= 'seleccionar '	
FILTRAR ::= 'filtrar '	DONDE ::= 'donde '
CREAR ::= 'crear '	RENOMBRAR ::= 'renombrar '
ORDENAR ::= 'ordenar '	POR ::= 'por '
ASC ::= 'asc '	DESC ::= 'desc '
ELIMINAR ::= 'eliminar '	DUPLICADOS ::= 'duplicados '
NULOS ::= 'nulos '	RELLENAR ::= 'rellenar '
CONVERTIR ::= 'convertir '	
AGRUPAR ::= 'agrupar '	RESUMIR ::= 'resumir '
LIMITAR ::= 'limitar '	GRAFICAR ::= 'graficar '
BARRAS ::= 'barras '	LINEAS ::= 'lineas '
HISTOGRAMA ::= 'histograma '	
DISPERSION ::= 'dispersion '	
CAJA ::= 'caja '	EJE_X ::= 'eje_x '
EJE_Y ::= 'eje_y '	TITULO ::= 'titulo '
LEYENDA ::= 'leyenda '	
T_NUMERO ::= 'numero '	T_DECIMAL ::= 'decimal '
T_TEXTO ::= 'texto '	T_BOOLEANO ::= 'booleano '

9.2. Literales y operadores

BOOLEANO ::= 'verdadero ' | 'falso '
 NULO ::= 'nulo '

FLECHA ::= '>'
 AND ::= '&&'
 OR ::= '||'
 IGUAL ::= '=='
 DIFERENTE ::= '!='

MAYOR_IGUAL ::= '>='
MENOR_IGUAL ::= '<='

9.3. Tokens generales

DECIMAL ::= DIGITO+ '.' DIGITO+
NUMERO ::= DIGITO+
TEXTO ::= '"' (~["\r\n] | '\\"')* '"'
IDENTIFICADOR ::= LETRA (LETRA | DIGITO | '_')*

LETRA ::= [a-zA-ZáéíóúñÁÉÍÓÚÑ] (incluye vocales acentuadas y n con tilde)

DIGITO ::= [0-9]

9.4. Elementos descartados

NL ::= ('\r'? '\n')+
WS ::= [\t]+ → descartado
COMENTARIO_LINEA ::= '#' ~["\r\n"]* → descartado
COMENTARIO_BLOQUE ::= '/-' '*?' '-/' → descartado

Los espacios, las tabulaciones y los comentarios se descartan: el parser nunca los ve. El salto de línea no se descarta, porque termina las sentencias.

La secuencia '\r'? '\n' contempla los archivos guardados en Windows, que terminan cada línea con retorno de carro y avance. El + final agrupa varios saltos consecutivos en un solo token, de modo que las líneas en blanco no generan ruido.

El delimitador de comentario de bloque es /- -/ y no el habitual /* */ para dejar libre el asterisco: /* podría confundirse con una división seguida de una multiplicación al leer el código.

10. Resolución de conflictos léxicos

Un mismo fragmento de texto puede encajar en varias reglas. El lexer de ANTLR resuelve el conflicto con dos criterios aplicados en orden.

Primero, la coincidencia más larga. Ante >=, tanto la regla '>' como MAYOR_IGUAL son candidatas, pero la segunda consume dos caracteres y gana. Lo mismo ocurre con 3.14: encaja con NUMERO tomando solo el 3, y con DECIMAL tomando los cuatro caracteres; gana DECIMAL.

Segundo, en caso de empate, la regla declarada antes. La palabra fijo encaja con FIJO y con IDENTIFICADOR, y ambas consumen cuatro caracteres. El empate lo resuelve el orden de declaración, y por eso todas las palabras reservadas se escriben antes que IDENTIFICADOR.

La consecuencia práctica es que el orden de las reglas léxicas no es cosmético. Si la regla de los identificadores se declarara primero, ninguna palabra reservada se reconocería como tal y el lenguaje dejaría de funcionar por completo.

11. Verificación

La gramática se compila con:

```
java -jar antlr-4.13.2-complete.jar \
    -Dlanguage=Python3 -visitor -no-listener \
    -o generado gramaticas/Terra.g4
```

La compilación termina sin errores y sin advertencias de ambigüedad, lo que confirma que ninguna entrada válida admite dos árboles de análisis distintos. Se generan tres archivos: el lexer, el parser y la clase base del visitor, en **gramaticas/**.

Para inspeccionar el árbol de un programa concreto:

```
./arbol.sh ejemplos/03_control.terra          # arbol en texto
./arbol.sh ejemplos/03_control.terra -gui      # arbol en ventana
python src/terra.py archivo.terra --tokens    # lista de tokens
python src/terra.py archivo.terra --validar    # veredicto de validez
```

Nueve de las cien pruebas de la suite verifican la gramática de forma directa: comprueban que los programas correctos se aceptan, que los incorrectos se rechazan, y que las tuberías y la instrucción **graficar** se reconocen aunque todavía no se ejecuten.